Expresiones regulares

Patrones para buscar texto y comprobar formatos

Buscar: `SOL` encuentra esa secuencia dentro de un texto. **Validar:** `^SOL$` exige esa palabra completa en una entrada de una sola línea.

01 / Piezas que puedes combinar

ELEMENTO	SIGNIFICADO	EJEMPLO DE PATRÓN
`^ / $`	Inicio / final de la entrada	`^SOL$`
`.`	Un carácter; normalmente no un salto de línea	`^A.B$`
`[ABC]`	Un carácter de la lista	`^[ABC]$`
`[A-C] / [0-9]`	Un carácter del intervalo	`^[A-C][0-9]$`
`[^0-9]`	Un carácter que no sea un dígito 0-9	`^[^0-9]$`
`?`	Cero o una repetición	`^SI!?$`
`*`	Cero o más repeticiones	`^AB*$`
`+`	Una o más repeticiones	`^AB+$`
`{3}`	Exactamente tres repeticiones	`^[0-9]{3}$`
`{2,4}`	Entre dos y cuatro repeticiones	`^[0-9]{2,4}$`
`{2,}`	Dos o más repeticiones	`^[0-9]{2,}$`
`(A|B)`	Agrupar alternativas: A o B	`^(SOL|LUNA)-[0-9]{2}$`
`\. / [.]`	Punto literal	`^AB[.][0-9]{2}$`

02 / Ejemplos de formato

DATO	PATRÓN Y EJEMPLO
Alias	`^[a-z]{3,8}$` Sí: ana / No: al (demasiado corto)
Color	`^(rojo|azul|verde)$` Sí: azul / No: amarillo
Aula	`^[A-Z][0-9]$` Sí: B3 / No: B12 (sobran caracteres)

Cómo leer el alias: `^` marca el inicio; `[a-z]` permite una letra de a a z; `{3,8}` pide entre tres y ocho; `$` marca el final. Los ejemplos presuponen comparación sensible a mayúsculas.

Referencias: Sintaxis ICU

REGEXP_LIKE en MySQL

Sintaxis, opciones y ejemplos de consulta

```
REGEXP_LIKE(texto_o_columna, 'patron' [, 'opciones'])
```

Los corchetes de esta notación indican un argumento opcional: **no se escriben**. Devuelve **1** si encuentra coincidencia, **0** si no y **NULL** si el texto o el patrón es NULL.

03 / Las cinco opciones de match_type

OPCIÓN	EFECTO	EJEMPLO Y RESULTADO
`c`	Distingue mayúsculas y minúsculas.	`REGEXP_LIKE('a', 'A', 'c') = 0`
`i`	Ignora mayúsculas y minúsculas.	`REGEXP_LIKE('a', 'A', 'i') = 1`
`m`	^ y $ también actúan por líneas.	`REGEXP_LIKE('A\nB', '^B$', 'm') = 1`
`n`	El punto . también acepta saltos de línea.	`REGEXP_LIKE('A\nB', '^A.B$', 'n') = 1`
`u`	Solo LF cuenta como fin de línea para ., ^ y $.	`REGEXP_LIKE('A\rB', '^B$', 'mu') = 0`

En estas cadenas SQL, `\n` es LF y `\r` es CR. En el último ejemplo, usar solo `'m'` daría 1. Combina opciones en una cadena: `'im'`. Si `c` e `i` se contradicen, manda la última.

```
SELECT REGEXP_LIKE('ana', '^[a-z]{3,8}$', 'c');
-- Resultado: 1
```

Sin tercer argumento: la sensibilidad a mayúsculas depende de la collation. Omitir `'c'` no garantiza que se rechacen las minúsculas. Una columna con `utf8mb4_0900_as_cs` sí las distingue.

04 / Del patrón a una cadena SQL

CONTEXTO	PUNTO LITERAL	EJEMPLO QUE CUMPLE
Patrón sin comillas	`^AB\.[0-9]{2}$`	`AB.27`
Cadena escrita en SQL	`'^AB\\.[0-9]{2}$'`	`AB.03`
Alternativa sin barras	`'^AB[.][0-9]{2}$'`	`AB.99`

En SQL, usa comillas simples y duplica la barra invertida para que llegue al motor regex. Esto supone que `NO_BACKSLASH_ESCAPES` está desactivado. `[0-9]` limita los dígitos a 0-9; `\d` admite también dígitos Unicode.

Entradas multilínea: $ puede coincidir antes del salto final. Para extremos absolutos usa `\A` y `\z`; en una cadena SQL: `'\\A[0-9]{4}\\z'`.

Referencias: REGEXP_LIKE · Cadenas SQL

CHECK con REGEXP_LIKE

Convierte el patrón en una regla de la tabla

05 / Plantilla que debes recordar

```
CHECK (REGEXP_LIKE(columna, 'patron'))

-- Con nombre y una opcion de comparacion:
CONSTRAINT nombre CHECK (REGEXP_LIKE(columna, 'patron', 'c'))
```

Columna: sin comillas simples. **Patrón y opciones:** entre comillas simples. **Paréntesis:** cierra tanto REGEXP_LIKE como CHECK. `CONSTRAINT nombre` es opcional.

06 / Tabla completa: sin tercer argumento

```
CREATE TABLE perfiles (
  alias VARCHAR(8)
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_as_cs
    NOT NULL,
  CONSTRAINT ck_alias
    CHECK (REGEXP_LIKE(alias, '^[a-z]{3,8}$'))
);
```

La columna ya distingue mayúsculas gracias a su collation. Por eso el CHECK puede usar **solo dos argumentos** y rechazar Ana. Para fijar esta comparación desde la función, puedes añadir `'c'`.

VALOR QUE SE INTENTA GUARDAR	RESULTADO
`'ana'`	Aceptado: entre tres y ocho letras de a a z.
`'al'`	Rechazado por CHECK: solo tiene dos letras.
`'Ana'`	Rechazado por CHECK: contiene una mayúscula.
`NULL`	Rechazado por NOT NULL.

07 / Si la tabla ya existe

```
ALTER TABLE perfiles
  ADD CONSTRAINT ck_alias
  CHECK (REGEXP_LIKE(alias, '^[a-z]{3,8}$'));
```

Es una alternativa al CREATE TABLE anterior: úsala cuando la restricción aún no exista. Las filas ya guardadas también deben cumplirla.

CHECK y NULL: CHECK rechaza una condición falsa, pero permite UNKNOWN (NULL). Añade NOT NULL si el campo es obligatorio. **Versión:** los CHECK se aplican desde MySQL 8.0.16; antes se analizaban y se ignoraban.

Referencias: Restricciones CHECK · Compatibilidad 8.0